Atividade Prática: Introdução à Linguagem C – Estrutura de Repetição e Reutilização de Código

📌 **Objetivo:**
O objetivo dessa atividade é ensinar aos alunos os conceitos de **estruturas de repetição** e **reutilização de código** na **linguagem C**, por meio de **loops** e **funções**. O aluno aprenderá a usar as estruturas de repetição, como o `for`, `while` e `do-while`, além de entender como reutilizar funções para aumentar a eficiência e a legibilidade do código.

📌 **Formato: Atividade Individual**
📌 **Duração: 2 horas**
📌 **Ferramentas: Compilador de C** (Dev C++, Code::Blocks, ou IDE online como replit.com)
📌 **Entrega:** Código em C com explicações sobre o uso das estruturas de repetição e funções, e boas práticas de programação.

📖 Contextualização e Situação Problema:

Você foi contratado para criar um programa para um **site de vendas de ingressos** de cinema. O sistema precisa:

- **Calcular o valor total** dos ingressos vendidos.
- **Exibir a quantidade total de ingressos vendidos**.
- **Reutilizar o código** para registrar diferentes sessões de filmes.
- O programa deve **permitir o cadastro de vários filmes** com o valor dos ingressos e o número de ingressos vendidos por cada sessão.

Você deve usar **estruturas de repetição** para **perguntar ao usuário** se deseja registrar mais sessões de filmes. Além disso, o código deve ser **reutilizável** para diferentes filmes, utilizando **funções** para evitar a repetição do mesmo código.

📋 Passo a Passo de Execução:

1. Preparação do Ambiente (10 minutos):

- Abra seu **compilador de C** (Dev C++, Code::Blocks ou IDE online).

- Crie um **novo arquivo** de código em C e salve com o nome `venda_ingressos.c`.

2. Estrutura Básica do Programa (30 minutos):

Comece criando a estrutura básica do programa, incluindo as bibliotecas necessárias e a função principal (`main`). O código a seguir solicita o nome do filme, o valor do ingresso e a quantidade de ingressos vendidos.

Código Inicial:

```C/C++
#include <stdio.h>  // Biblioteca para entrada e saída de dados

// Função para calcular o valor total dos ingressos vendidos
float calcularTotal(float preco, int quantidade) {
    return preco * quantidade;
}

int main() {
    char nomeFilme[50];
    float precoIngresso;
    int quantidadeIngressos, opcao;
    float totalGeral = 0;

    do {
        // Entrada de dados
        printf("Digite o nome do filme: ");
        scanf("%s", nomeFilme);

        printf("Digite o valor do ingresso: ");
        scanf("%f", &precoIngresso);

        printf("Digite a quantidade de ingressos vendidos: ");
        scanf("%d", &quantidadeIngressos);

        // Calcular total para o filme atual
        float total = calcularTotal(precoIngresso,
quantidadeIngressos);
```

```
        totalGeral += total;  // Somando ao total geral

        // Exibir resultado
        printf("Filme: %s\n", nomeFilme);
        printf("Valor total dos ingressos vendidos: R$ %.2f\n",
total);

        // Pergunta se o usuário deseja adicionar outro filme
        printf("\nDeseja registrar outro filme? (1 para sim, 0 para
não): ");
        scanf("%d", &opcao);

    } while (opcao == 1);  // Se o usuário escolher 1, o loop
continua

    // Exibe o total geral de vendas
    printf("\nTotal de vendas: R$ %.2f\n", totalGeral);

    return 0;
}
```

Explicação do Código:

1. **Biblioteca `stdio.h`:** Usada para entrada e saída de dados.

2. **Função `calcularTotal`:** Calcula o valor total dos ingressos vendidos, multiplicando o preço pelo número de ingressos vendidos. Essa função **reutiliza o código**, permitindo calcular o total de vendas para diferentes filmes.

3. **Estrutura de repetição `do-while`:** Usada para permitir que o usuário cadastre **múltiplos filmes**. O loop pergunta ao usuário se deseja registrar outro filme (opção 1 para continuar, opção 0 para encerrar).

3. Explicação das Estruturas de Repetição (30 minutos):

- **`while`**: A estrutura de repetição `while` executa um bloco de código enquanto uma condição for verdadeira. A condição é verificada antes de cada iteração.

Exemplo de `while`:

```
C/C++
while (condicao) {
    // Código a ser repetido enquanto a condição for verdadeira
}
```

- **`do-while`**: A estrutura `do-while` é semelhante ao `while`, mas verifica a condição após a execução do bloco de código. Isso significa que o código dentro do loop será **executado pelo menos uma vez**, mesmo que a condição seja falsa inicialmente.

  Exemplo de `do-while`:

```
C/C++
do {
    // Código a ser repetido
} while (condicao);
```

4. Melhorando o Programa com Reutilização de Código (30 minutos):

Agora, adicione a **função `calcularTotal`** para **evitar a repetição de código**. Isso é importante quando o código pode ser usado em várias partes do programa, como neste caso, em que o cálculo do valor total dos ingressos se repete.

Função `calcularTotal`:

```
C/C++
float calcularTotal(float preco, int quantidade) {
    return preco * quantidade;
}
```

5. Desafio Adicional (30 minutos):

Desenvolva um programa que não apenas registre filmes, mas também faça a **média do valor dos ingressos vendidos** e mostre o **filme com maior quantidade de ingressos vendidos**.

Exigências:

- Registre pelo menos **5 filmes** com o valor do ingresso e a quantidade de ingressos vendidos.
- Calcule e mostre a **média do valor total das vendas**.
- Exiba o **filme com maior quantidade de ingressos vendidos**.

Exemplo de código para o desafio adicional:

C/C++

```
#include <stdio.h>  // Biblioteca para entrada e saída de dados

float calcularTotal(float preco, int quantidade) {
    return preco * quantidade;
}

int main() {
    char nomeFilme[50];
    float precoIngresso, totalFilme, totalGeral = 0, mediaVendas =
0;
    int quantidadeIngressos, opcao, maxIngressos = 0, maxFilmeIndex
= 0;

    for (int i = 0; i < 5; i++) {
        printf("\nDigite o nome do filme: ");
        scanf("%s", nomeFilme);

        printf("Digite o valor do ingresso: ");
        scanf("%f", &precoIngresso);

        printf("Digite a quantidade de ingressos vendidos: ");
        scanf("%d", &quantidadeIngressos);

        totalFilme = calcularTotal(precoIngresso,
quantidadeIngressos);
        totalGeral += totalFilme;  // Soma total das vendas

        if (quantidadeIngressos > maxIngressos) {
```

```
            maxIngressos = quantidadeIngressos;
            maxFilmeIndex = i;
        }

        mediaVendas = totalGeral / (i + 1);  // Calculando a média
das vendas

        printf("Filme: %s\n", nomeFilme);
        printf("Total das vendas: R$ %.2f\n", totalFilme);
    }

    // Exibe os resultados finais
    printf("\nTotal geral das vendas: R$ %.2f\n", totalGeral);
    printf("Média das vendas: R$ %.2f\n", mediaVendas);
    printf("Filme com maior quantidade de ingressos vendidos: Filme
%d com %d ingressos\n", maxFilmeIndex + 1, maxIngressos);

    return 0;
}
```

📄 Formato de Entrega:

1. **Cabeçalho no Script:** O arquivo deve conter um cabeçalho com o nome do programa, versão, autor, data e descrição.

Cabeçalho Exemplo:

C/C++
```
// Programa: Sistema de Controle de Vendas de Ingressos
// Versão: 1.0
// Autor: [Nome do Estudante]
// Data: 25/03/2025
// Descrição: Programa que registra as vendas de ingressos de
cinema, calcula total de vendas e realiza outras operações de
cálculo.
```

2. **Código Completo:** O script em C deve ser enviado no formato **.txt** (Bloco de Notas) contendo:
    - A estrutura básica de entrada e saída.
    - A implementação das funções e loops conforme descrito.
    - Comentários explicativos sobre as partes do código.
3. **Relatório Explicativo:** O relatório deve incluir uma breve explicação sobre o uso das **estruturas de repetição**, **funções** e **como o código foi estruturado**.

📋 Rubrica de Avaliação:

Critério	Descrição	Peso (%)
Correção do Código	O código executa corretamente as operações de repetição e reutilização de código.	40%
Boas Práticas de Programação	O código segue boas práticas (comentários, indentação, nomes significativos).	30%
Organização e Clareza	O código está bem estruturado, com boa legibilidade e clareza.	20%
Testes e Resultados	O programa foi testado adequadamente e produziu os resultados esperados.	10%

🎯 Resumo da Atividade:

- **Objetivo:** Ensinar o uso de **estruturas de repetição** e **reutilização de código** na **linguagem C**.
- **Conceitos: Loops**, **funções**, **entrada e saída de dados**, **boas práticas de programação**.
- **Ferramenta:** Uso de **Bloco de Notas** para entrega do código.

Esta atividade permitirá que os alunos compreendam os conceitos essenciais de **programação em C** e como aplicar **estruturas de repetição** e **funções** para tornar o código mais modular e eficiente.